SCALE FOR PROJECT PUSH_SWAP (/PROJECTS/42NEXT-PUSH_SWAP)

You should evaluate 2 students in this team

★

Git repository

```
git@vogsphere.42antananarivo.mg:vogsphere/intra-uuid-22827ae3-
```

Introduction

Please comply with the following rules:

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Identify possible dysfunctions in the evaluated student's or group's project. Take the time to discuss and debate the problems that may have been identified.

- You must consider that there might be some differences in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade them as honestly as possible. Pedagogy is effective only if peer evaluation is taken seriously.

Guidelines

- Only grade the work that was turned in the Git repository of the evaluated student or group.

- Ensure that the Git repository belongs to the student(s) and that the project is the expected one. Additionally, ensure that 'git clone' is executed in an empty folder.

- Check carefully that no malicious aliases were used to fool you and make you evaluate something that is not the content of the official repository.

- To avoid any surprises and if applicable, review together any scripts used to facilitate the grading (scripts for testing or automation).

- If you have not completed the assignment you are going to evaluate, you have to read the entire subject prior to starting the evaluation process.

- Use the available flags to report an empty repository, a non-functioning program, a Norm error, cheating, and so forth.
In these cases, the evaluation process ends and the final grade is 0, or -42 in case of cheating. However, except for cheating, students are strongly encouraged to review together the work that was submitted, in order to identify any mistakes that shouldn't be repeated in the future.

- You must also verify the absence of memory leaks. Any memory allocated on the heap must be properly freed before the end of execution.
You are allowed to use any of the different tools available on the computer, such as leaks, valgrind, or e_fence. If memory leaks are detected, select the appropriate flag.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/201834/en.subject.pdf)

fedora_checker (https://cdn.intra.42.fr/document/document/47390/fedora_checker)

checker_Mac (https://cdn.intra.42.fr/document/document/47391/checker_Mac)

checker_linux (https://cdn.intra.42.fr/document/document/47392/checker_linux)

Preliminaries

Before starting the evaluation, ensure that you have read and understood the entire subject.

group

The review is done in the presence of BOTH learners being graded.
This is how everyone progresses: by interacting with others.

- Both learners must be present during the defense.
- No report: 0, the review is over.
- As soon as an exercise is non-functional, the review stops. You can look at the code of the f exercises, but they will not be graded.

Yes No

verifications

Group project verification:

- Verify that exactly 2 learners are listed as contributors in the repository.
- Check that both learners can explain and defend any part of the implemented code.
- Confirm that the README.md clearly documents each learner's contributions.
- Both learners must demonstrate understanding of the entire codebase, not just their parts. I requirements are not met, the evaluation fails.

Yes No

README.md Compilance check

Does the repository contain a README.md file at its root, and does it include all of the following?

- The first line is italicized and formatted exactly as: *This activity has been created as part of i by <login1>, <login2>* (exactly 2 learners required).
- A "Description" section explaining the activity's purpose and providing a brief overview.
- An "Instructions" section with relevant details about compilation, installation, and/or executic
- A "Resources" section listing references (documentation, tutorials, etc.) and explaining how specifying for which tasks and which parts of the project.
- A detailed explanation and justification of the algorithms selected for this activity (Simple O(O(n√n), Complex O(n log n), and Adaptive strategies).
- A clear documentation of each learner's contributions to the project.

Yes No

Mandatory part

Reminder: During the defense, no segmentation faults, unexpected crashes, premature terminatio uncontrolled exits are allowed. Otherwise, the final grade will be 0. Use the appropriate flag. This r throughout the entire defense.

Norminette

Run the Norminette. If there is an error, the evaluation stops here.
You can keep going and discuss the implementation of the code, but the assignment will not be graded.

Yes No

Compilation

Check that a Makefile is present and contains the usual rules: NAME, all, clean, fclean, re.
The Makefile must compile the project with the flags -Wall -Wextra -Werror and must not relink.
Test the compilation by running 'make' and verify that the push_swap executable is created.
Test that 'make clean', 'make fclean', and 'make re' work correctly.
If there is a compilation error or the Makefile doesn't work properly, the evaluation stops here.
You can keep going and discuss the implementation of the code, but the assignment will not be graded.

Yes No

Memory leaks check

Throughout the defense, pay attention to the amount of memory used by push_swap (using the command top for example) in order to detect any anomalies and ensure that allocated memory is properly freed. If there are significant memory leaks, the final grade is 0.

Note: Minor memory issues may be acceptable depending on the implementation, but major leaks or crashes should result in failure.

Yes No

Error management

In this section, we'll evaluate push_swap's error management.
If most of these tests fail, no points will be awarded for this section.

Note: At least 3 out of 4 error cases should be handled correctly.

- Run push_swap with non numeric parameters. The program must display "Error" followed by a '\n' on the standard error.
- Run push_swap with a duplicate numeric parameter. The program must display "Error" followed by a '\n' on the standard error.
- Run push_swap with only numeric parameters including one greater than MAXINT. The program must display "Error" followed by a '\n' on the standard error.
- Run push_swap without any parameters. The program must not display anything and give the prompt back.

Yes No

Strategy Selection - Basic Tests

Test the strategy selection flags. If most of these tests fail, no points will be awarded for this section.

- Run "$>./push_swap --simple 5 4 3 2 1" and verify it produces valid output that sorts the numbers.
- Run "$>./push_swap --medium 5 4 3 2 1" and verify it produces valid output that sorts the numbers.
- Run "$>./push_swap --complex 5 4 3 2 1" and verify it produces valid output that sorts the numbers.
- Run "$>./push_swap --adaptive 5 4 3 2 1" and verify it produces valid output that sorts the numbers.
- Verify that running without any flag defaults to --adaptive behavior.

Note: At least 3 out of 5 tests should work for this section to pass.

Yes No

Identity test - Already sorted inputs

Test push_swap's behavior with already sorted inputs.
If most tests fail, no points will be awarded for this section.

- Run "$>./push_swap 42". The program should display nothing.
- Run "$>./push_swap 2 3". The program should display nothing.
- Run "$>./push_swap 0 1 2 3". The program should display nothing.
- Run "$>./push_swap 0 1 2 3 4 5 6 7 8 9". The program should display nothing.

All tests should produce no output (0 instructions) since the inputs are already sorted.
Note: At least 3 out of 4 tests should work correctly.

Yes No

Small inputs (3 numbers)

Test with 3 numbers. Use the checker binary provided.

- Run "$>ARG="2 1 0"; ./push_swap $ARG | ./checker_linux $ARG". Check that the checker displays "OK" and that the number of instructions is reasonable (≤5 is acceptable, ≤3 is good).
- Test with other 3-number combinations like "0 2 1" or "1 0 2" and verify the checker displays "OK" with reasonable instruction count.

Note: Some variation in instruction count is normal. Focus on correctness first.

Yes No

Medium inputs (5 numbers)

Test with 5 numbers. Use the checker binary provided.

- Run "$>ARG="1 5 2 4 3"; ./push_swap $ARG | ./checker_linux $ARG". Check that the checker displays "OK" and that the number of instructions is reasonable (≤15 is acceptable, ≤12 is good).
- Test with 2-3 other combinations of 5 random numbers. Check that the checker displays "OK" and instruction count is reasonable. Example: try "5 1 4 2 3" or "3 5 1 4 2"

Note: Different algorithms may produce different instruction counts. Correctness is more important than perfect optimization.

Yes No

Benchmark Mode and Disorder Calculation

Test the benchmark mode functionality. This is not a failing requirement but should be implemented according to the subject.

- Run "$>./push_swap --bench --simple 5 4 3 2 1 2>/dev/null" and verify it produces sorting instructions on stdout.
- Run "$>./push_swap --bench --simple 5 4 3 2 1 2>bench.txt >/dev/null && cat bench.txt" and verify the benchmark output contains most of:
  - Disorder percentage (should be present)
  - Strategy name and complexity class
  - Total operations count
  - Individual operation counts for operations
- Test disorder calculation with known inputs:
  - For sorted input "1 2 3 4 5", disorder should be close to 0.00%
  - For reverse sorted input "5 4 3 2 1", disorder should be close to 100.00%

Note: Minor variations in output format are acceptable if the core information is present.

Yes No

Large inputs (100 numbers)

Test with 100 random numbers. Use the checker binary provided.
Generate random numbers using: shuf -i 1-500 -n 100

- Run the test 2-3 times with different random sets.
- Check that the checker displays "OK" for all tests
- The program should use less than 2000 operations to pass this test.
- Less than 1500 is good, less than 700 is excellent performance.

Note: Some variation between runs is normal. Focus on overall correctness.

Yes No

Code review and algorithm explanation

Ask the learner to briefly explain their approach:

- How does the --simple strategy work? (Should be $O(n^2)$ approach)
- How does the --medium strategy work? (Should be $O(n\sqrt{n})$ approach)
- How does the --complex strategy work? (Should be $O(n \log n)$ approach)

- How does the --adaptive strategy choose which method to use?

The learner should be able to give a basic explanation of their algorithms.
Don't expect perfect theoretical analysis - focus on practical understanding.
It's okay if they can't explain every detail perfectly.

Yes No

Strategy flags testing

Test different strategy flags with the same input:

- Generate 50 random numbers: shuf -i 1-200 -n 50
- Test the same input with --simple, --medium, --complex flags
- Most should produce valid output that sorts correctly
- Generally, --complex should use fewer instructions than --simple
- The --adaptive flag (or no flag) should work and choose automatically

Note: It's acceptable if not all strategies are perfectly optimized.
The important thing is that they work and show some performance differences.

Yes No

Very large inputs (500 numbers)

Test with 500 random numbers. Use the checker binary provided.
Generate using: shuf -i 1-1000 -n 500

- Run the test 2 times with different random sets
- Check that the checker displays "OK" for both tests
- The program should use less than 12000 operations to pass this test.
- Less than 8000 is good, less than 5500 is excellent performance.

Note: This is a challenging test. Some variation in performance is expected.
Focus on correctness first, then performance

Yes No

Quick live coding modification

Ask the reviewee to add a new flag "--count-only" to their push_swap program that only displays the total number of operations needed to sort the stack, without showing the actual operations.
For example, "./push_swap --count-only 3 2 1" should output just "3" instead of the full list of operations.

The reviewee should be able to locate the relevant parsing and output code, make the necessary modifications, and demonstrate that it works with a few test cases.
The entire task, including the demonstration, should take no more than 10 minutes. Was this procedure followed and did everything work correctly?

Yes No

Bonus

Reminder : Remember that for the duration of the defence, no segfault, nor other unexpected, pre uncontrolled or unexpected termination of the program, else the final grade is 0. Use the appropria is active throughout the whole defence. We will look at your bonuses if and only if your mandatory EXCELLENT. This means that you must complete the mandatory part, beginning to end, and your management needs to be flawless, even in cases of twisted or bad usage.If the mandatory part do perfect score during this defense, the bonus section will be completely ignored.

Checker program - Error management

In this section, we'll evaluate the checker's error management.
If at least one fails, no points will be awarded for this section. Move to the next one.

- Run checker with non numeric parameters. The program must display "Error" followed by a '\n' on the standard error.
- Run checker with a duplicate numeric parameter. The program must display "Error" followed by a '\n' on the standard error.

- Run checker with only numeric parameters including one greater than MAXINT. The program must display "Error" followed by a '\n' on the standard error.
- Run checker without any parameters. The program must not display anything and give the prompt back.
- Run checker with valid parameters, and write an action that doesn't exist during the instruction phase. The program must display "Error" followed by a '\n' on the standard error.
- Run checker with valid parameters, and write an action with one or several spaces before and/or after the action during the instruction phase. The program must display "Error" followed by a '\n' on the standard error.

☑ Yes ☒ No

Checker program - False tests

In this section, we'll evaluate the checker's ability to manage a list of instructions that doesn't sort the list. Execute the following 2 tests. If at least one fails, no points will be awarded for this section. Move to the next one.

Don't forget to press CTRL+D to stop reading during the instruction phase.

- Run checker with the following command "$>./checker 0 9 1 8 2 7 3 6 4 5" then write the following valid action list "[sa, pb, rrr]". The checker should display "KO".
- Run checker with a valid list as parameter of your choice then write a valid instruction list that doesn't order the integers.The checker should display "KO". You'll have to specifically check that the program wasn't developed to only answer correctly on the test included in this scale. You should repeat this test couple of times with several permutations before you validate it.

☑ Yes ☒ No

Checker program - Right tests

In this section, we'll evaluate the checker's ability to manage a list of instructions that sort the list. Execute the following 2 tests. If at least one fails, no points will be awarded for this section. Move to the next one.

Don't forget to press CTRL+D to stop reading during the instruction phase.

- Run checker with the following command "$>./checker 0 1 2" then press CTRL+D without writing any instruction. The program should display "OK".
- Run checker with the following command "$>./checker 0 9 1 8 2" then write the following valid action list "[pb, ra, pb, ra, sa, ra, pa, pa]". The program should display "OK".
- Run checker with a valid list as parameter of your choice then write a valid instruction list that correctly orders the integers. Checker must display "OK". You'll have to specifically check that the program wasn't developed to only answer correctly on the test included in this scale. You should repeat this test couple of times with several permutations before you validate it.

☑ Yes ☒ No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work | Incomplete work | Invalid compilation | Norme | Cheat

Concerning situation | Leaks | Forbidden function | Can't support

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation